COMP4141

Theory of Computation

Lecture 12: **P** and **NP**

Administrivia

- Assignment 3A and 3B available: Due **Mon 11th April, 12:00 (AEST)**
  - Part A: Automarked – immediate feedback; hidden cases may increase before 1st April
  - Part B: Submission as usual
- Assignments 1 & 2 solutions posted this week

Outline

P and NP

Nearly all (non-)deterministic models of computation are time equivalent up to some polynomial. This motivates

Definition

$$\mathbf{P} = \bigcup_{k \in \mathbb{N}} \mathbf{TIME}(n^k)$$

Definition

$$\mathbf{NP} = \bigcup_{k \in \mathbb{N}} \mathbf{NTIME}(n^k)$$

Examples in **P**

s-t Reachability

Input: A directed graph G and vertices s, t

Question: Is t reachable from s in G?

RelPrime

Input: Natural numbers x, y

Question: Is $\gcd(x, y) = 1$?

Details are in [Sipser2006].

Examples in **P** cont.

Theorem

Every CFL is in **P**.

Examples in **P** cont.

Theorem

CFL Acceptance

Input: *A CFL L and word w*

Question: *Is $w \in L$?*

is in **P**

Examples in **P** cont.

Theorem

CFL Acceptance

Input: *A CFG G and word w*

Question: *Is $w \in L(G)$?*

is in **P**

Examples in **P** cont.

Theorem

CFL Acceptance

Input: *A CFG G and word w*

Question: *Is $w \in L(G)$?*

is in **P**

Proof.

For CFG G in CNF the CYK algorithm runs in $O(|w|^3)$ time on w. □

Examples in **NP**

HamPath

Input: A graph G (n vertices, m edges)

Question: Is there a path in G that visits every vertex exactly once?

Examples in **NP**

HamPath

Input: A graph G (n vertices, m edges)

Question: Is there a path in G that visits every vertex exactly once?

An **NP** algorithm:

$\mathtt{P} = \epsilon$
while $|\mathtt{P}| < n$:
  non-deterministically guess a vertex v
  if $v \in \mathtt{P}$ then REJECT
  if $|\mathtt{P}| > 0$ and $(\mathtt{last}(\mathtt{P}), v) \notin E$ then REJECT
  $\mathtt{P} = \mathtt{append}(\mathtt{P}, v)$
ACCEPT

Examples in **NP**

HamPath

Input: A graph G (n vertices, m edges)

Question: Is there a path in G that visits every vertex exactly once?

Another **NP** algorithm:

Non-deterministically guess a sequence of n vertices, `P`
If `P` contains duplicated vertices then REJECT
If `P` is not a path then REJECT
ACCEPT

Examples in **NP**

EulerPath

Input: A graph G (n vertices, m edges)

Question: Is there a path in G that visits every edge exactly once?

Examples in **NP**

EulerPath

Input: A graph G (n vertices, m edges)

Question: Is there a path in G that visits every edge exactly once?

An **NP** algorithm:

Non-deterministically guess a sequence of m edges, `P`
If `P` contains duplicated edges then REJECT
If `P` is not a path then REJECT
ACCEPT

Examples in **NP**

EulerPath

Input: A graph G (n vertices, m edges)

Question: Is there a path in G that visits every edge exactly once?

A **P** algorithm:

```
Compute the degree of each vertex
If G has more than two vertices of odd degree then
   REJECT
else
   ACCEPT
```

Beyond (?) **P**

- HamPath
- Factorization: Can be solved in quantum computing equivalent of polynomial time
- Halt: Cannot be decided

Halt in k steps

Input: Given a TM M and an integer k

Question: Does M halt in at most k steps?

Outline

Verifiers

Verifying an answer is often much easier than finding it.

Definition

A **verifier** for a language A is an algorithm V, where

$$A = \{\, w \mid V \text{ accepts } \langle w, c \rangle \text{ for some string } c \,\}$$

We measure the running time of a verifier only in terms of the length of w, not that of the **certificate** (or **proof**) c. Language A is **polynomially verifiable** if it has a polynomial time verifier.

NB

Certificates for a polynomially verifiable language need only have polynomial length. Such certificates are known as **short** *certificates.*

Verifiers: Examples

Examples

- For HAMPATH a certificate for $\langle G \rangle$ could be the sequence of nodes forming a Hamiltonian path in G.
- For COMPOSITES a certificate for $\langle x \rangle$ could be a non-trivial divisor of x.

NP and verifiers

NP is the class of languages that have polynomial time verifiers.

Theorem

A language is in **NP** *iff it is polynomially verifiable*

Proof.

"$\subseteq$:" use nondeterminism to guess the certificate.
"$\supseteq$:" use the accepting branch of the computation tree as certificate. □

Example: Cliques

A *clique* in an undirected graph is a fully connected subgraph. A k-clique is a clique with k nodes.

Clique

Input: An undirected graph G and an integer k

Question: Does G contain a k-clique?

is in **NP**.

Example: Cliques

Theorem

CLIQUE *is in* **NP**

Proof.

- Certificate: Set of vertices K
- Verifier:
  1. Check that K has k vertices
  2. Check that the vertices form a clique: for each pair of vertices $v_i, v_j \in K$, check that $(v_i, v_j) \in E$
- Verifier runs in polynomial time:
  - K has at most $n = |V|$ vertices.
  - Step 1 takes $O(n)$ time.
  - Step 2 takes $O(m)$ time for each vertex pair, so $O(n^2 m)$ time at worst.
  - Total time $O(n + n^2 m)$ which is polynomial in $|G| + |k|$

□

Outline

$\mathbf{P} \stackrel{?}{=} \mathbf{NP}$

Recall:
P—problems that can be solved in polynomial time on a TM.
NP—problems that can be solved in polynomial time on an NTM.

Question

Is $\mathbf{P} = \mathbf{NP}$*?*

Equivalently: Can we simulate a polynomial time non-deterministic TM (NTM) in polynomial time on a (deterministic) TM?

At this point, no one knows for sure, but "no" might be a good bet.

NP-complete problems

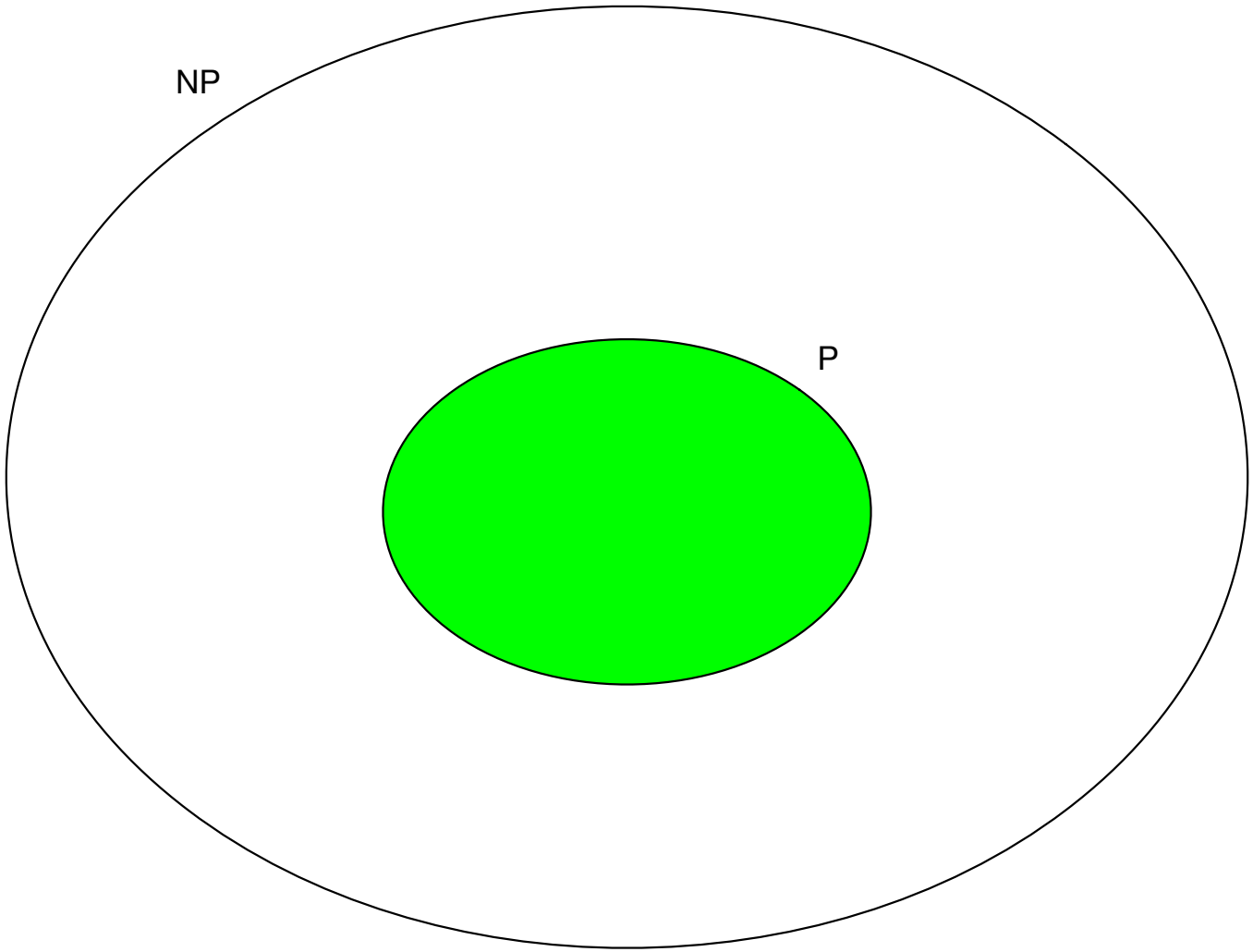
This is about decision problems (problems with yes/no answers).

Equivalently, solving the membership problem $x \in L$.

Obviously $\mathbf{P} \subseteq \mathbf{NP}$.

Nobody knows for sure whether $\mathbf{NP} \subseteq \mathbf{P}$

Intuitively, **NP**-complete problems are the "hardest" problems in **NP**.


NP
P

P Reducibility

Recall how we use mapping-reducibility to transfer (un)decidabilty from one problem to the next.

Definition

$f : \Sigma^* \longrightarrow \Sigma^*$ is a *polynomial time computable* (or **P** *computable*) function if some polynomial time TM M exists that halts with just $f(w)$ on its tape, when started on any input $w \in \Sigma^*$.

Definition

$A \subseteq \Sigma_1^*$ is *polynomial time mapping reducible* (or **P** *reducible*) to $B \subseteq \Sigma_2^*$, written $A \leq_{\mathbf{P}} B$, if a **P** computable function $f : \Sigma_1^* \longrightarrow \Sigma_2^*$ exists that is also a reduction (from A to B).

P Reducibility cont.

Theorem

If $A \leq_{\mathbf{P}} B$ *and* $B \in \mathbf{P}$ *then* $A \in \mathbf{P}$.

Proof.

Suppose f is the **P** reduction from A to B that runs in time $O(n^{k_1})$ and M_B is a **P** decider for B that runs in time $O(n^{k_2})$.

To decide $w \in A$, first compute $f(w)$, then run M_B on $f(w)$.

Note that the input to M_B has size at most $|w|^{k_1}$.

The total running time is $O(|w|^{k_1} + (|w|^{k_1})^{k_2}) = O(|w|^{k_1 k_2})$.

□

NP-Hardness

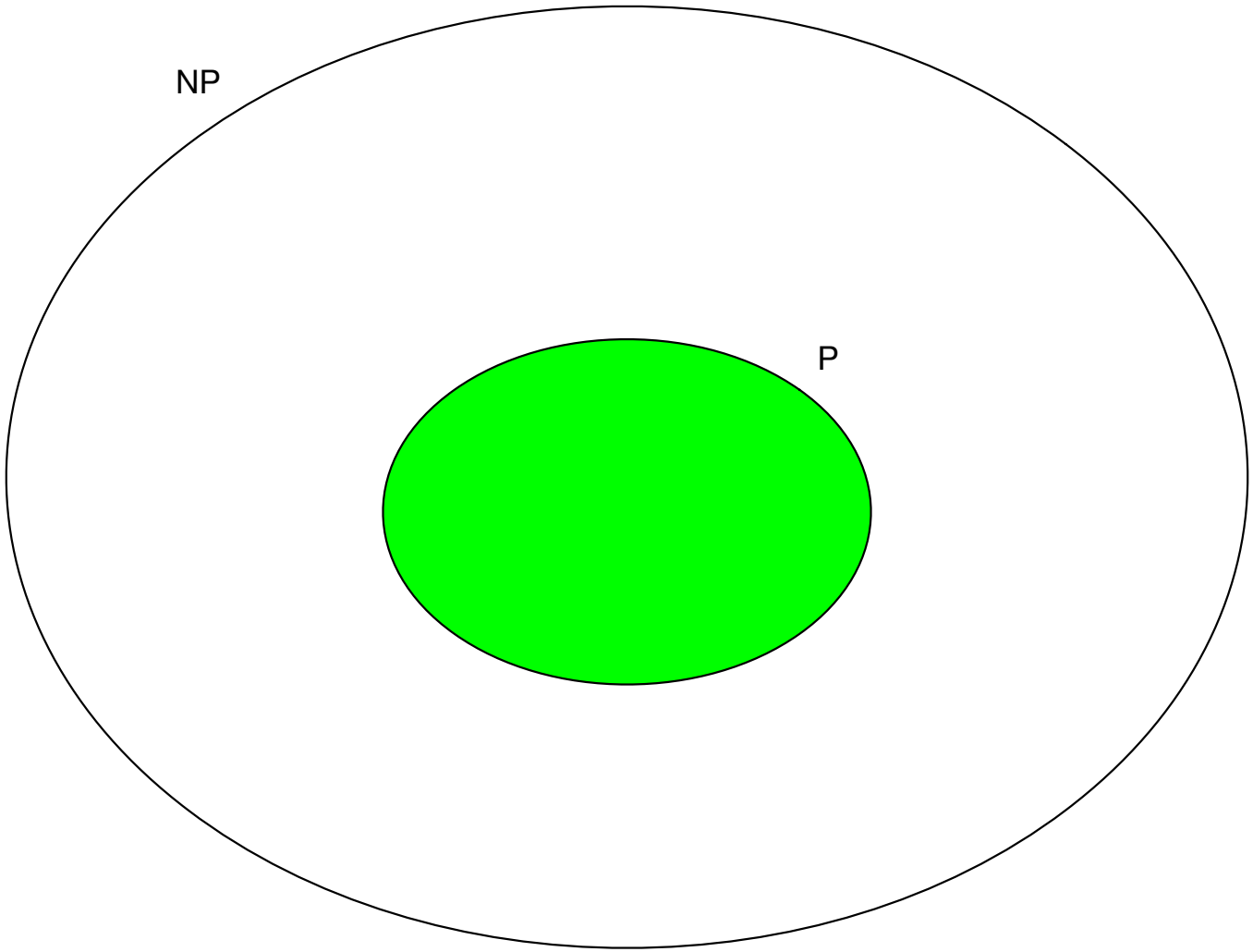
Definition

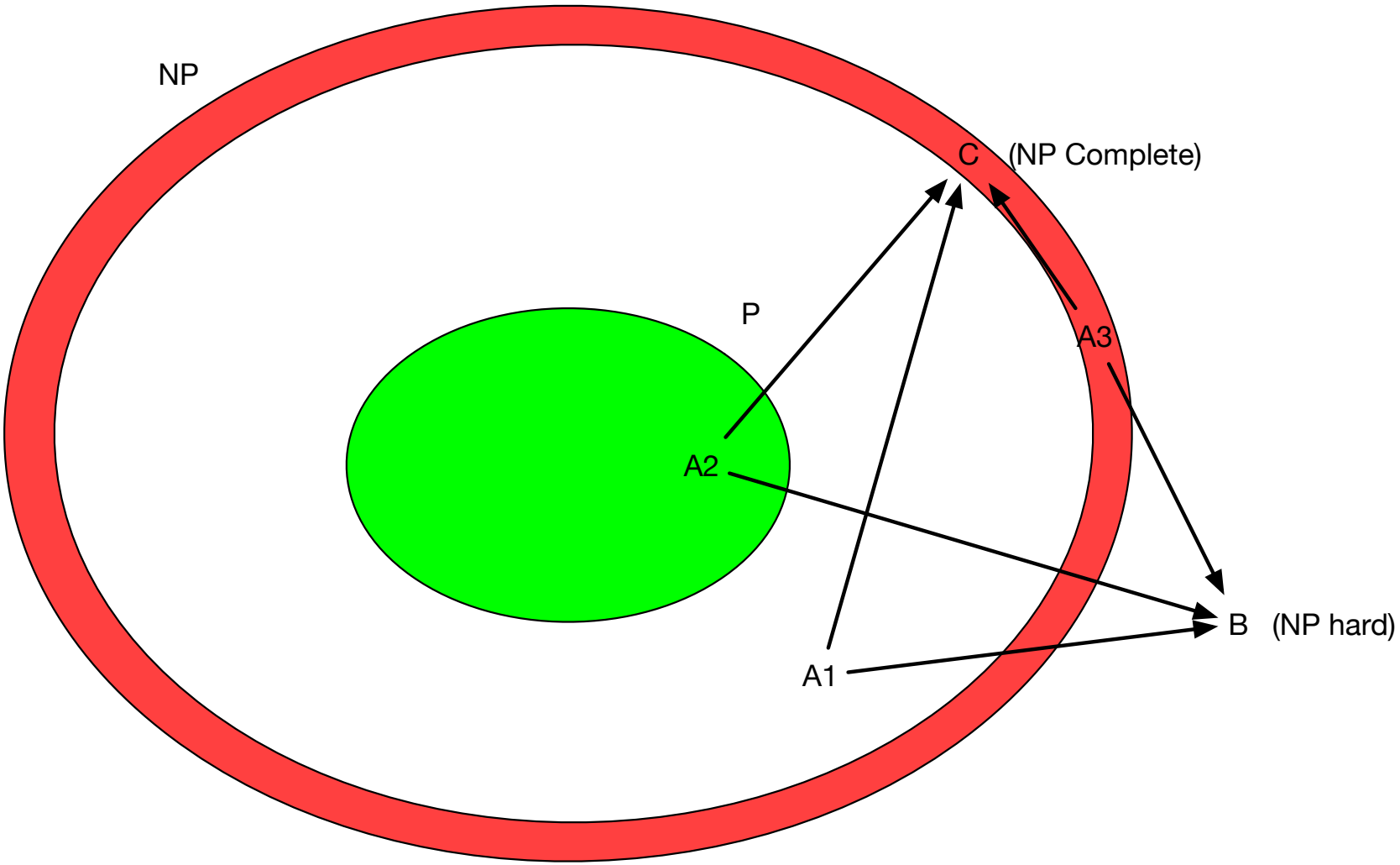
A language B is **NP**-*hard* if *every* $A \in \mathbf{NP}$ is **P** reducible to B.

Intuitively, this says B is at least as hard as any problem in **NP**.

Theorem

If B *is* **NP**-*hard and* $B \leq_{\mathbf{P}} C$ *then* C *is* **NP**-*hard.*


NP
P


NP
C
(NP Complete)
P
A3
A2
B
(NP hard)
A1

NP-Completeness

Definition

A language B is **NP**-*complete* if

1. $B \in \mathbf{NP}$
2. B is **NP**-hard (i.e. *every* $A \in \mathbf{NP}$ is **P** reducible to B).

Theorem

If B *is* **NP**-*complete and* $B \in \mathbf{P}$ *then* $\mathbf{P} = \mathbf{NP}$.

Theorem

If B *is* **NP**-*complete and* $B \leq_{\mathbf{P}} C$ *for* $C \in \mathbf{NP}$, *then* C *is* **NP**-*complete.*

Proof.

Polynomial time reductions compose. □

NP-Completeness

If there are any problems in **NP** $\setminus$ **P**, the **NP**-complete problems are all there.

Every **NP**-complete problem can be translated in deterministic polynomial time to every other **NP**-complete problem.

So, if there is a **P** solution to one **NP**-complete problem, there is a **P** solution to every **NP** problem.

NP-Hardness by Reduction

Typical method: Reduce a known **NP**-hard problem P_1 to the new problem P_2 .

Basic Proof Strategy

NP-completeness is a good news/bad news situation.

- Good news: The problem is in **NP**!
- Bad news: The problem is **NP**-hard!

So, a typical **NP**-completeness proof consists of two parts:

1. Prove that the problem is in **NP** (i.e., it has **P** verifier).
2. Prove that the problem is at least as hard as other problems in **NP**.

A TM can simulate an ordinary computer in polynomial time, so it is sufficient to describe a polynomial-time checking algorithm that will run on any reasonable model of computation.

NP-Hardness

A problem is **NP**-*hard* if having a polynomial-time solution to it would give us a polynomial solution to every problem in **NP**.

Proving that the problem is NP-hard: The usual strategy is to find a polynomial-time reduction of a known **NP**-hard problem (say P_1) to the problem in question (say P_2).

The goal is to show that P_2 is at least as hard (in terms of polynomial vs. super-polynomial time) as P_1.

Repeated warning: Make sure you are reducing the known problem to the unknown problem!

In practice, there are now thousands of known **NP**-complete problems. A good technique is to look for one similar to the one you are trying to prove **NP**-hard.

Computers and Intractability - A guide to theory of NP-completeness, M.R. Garey and D.S. Johnson, Freeman 1979 lists a whole bunch.

Outline

Boolean Formulae

Let $\mathit{Prop} = \{x, y, \ldots\}$ be a countable set of *Boolean variables* (or *propositions*).

A CFG for Boolean formulae over Prop is:

$$\varphi \to p \mid \varphi \wedge \varphi \mid \neg\varphi \mid (\varphi)$$
$$p \to x \mid y \mid \ldots$$

We use abbreviations such as

$$\varphi_1 \vee \varphi_2 = \neg(\neg\varphi_1 \wedge \neg\varphi_2) \qquad \varphi_1 \Rightarrow \varphi_2 = \neg\varphi_1 \vee \varphi_2$$
$$\textsc{false} = (x \wedge \neg x) \qquad \textsc{true} = \neg\textsc{false}$$

Let $\mathit{Prop}(\varphi)$ be the propositions that occur in φ.

Semantics of Boolean Formulae

A Boolean formula is either `true` or `false`, possibly depending on the interpretation of its propositions. Let $\mathbb{B} = \{\texttt{false}, \texttt{true}\}$.

Definition

An *interpretation* (of $Prop(\varphi)$) is a function $\pi : Prop(\varphi) \longrightarrow \mathbb{B}$.
For Boolean formulae φ we define π *satisfies* φ, written $\pi \models \varphi$, inductively by:
Base: $\pi \models x$ iff $\pi(x) = \texttt{true}$.
Induction:

- $\pi \models \neg\varphi$ iff $\pi \not\models \varphi$.
- $\pi \models \varphi_1 \wedge \varphi_2$ iff both $\pi \models \varphi_1$ and $\pi \models \varphi_2$.
- $\pi \models (\varphi)$ iff $\pi \models \varphi$.

φ is *satisfiable* if there exists an interpretation π such that $\pi \models \varphi$.

SAT—An **NP**-Complete Problem

SAT

Input: A Boolean formula φ

Question: Is φ satisfiable?

$$\mathrm{SAT} = \{ \langle \varphi \rangle \mid \varphi \text{ is a satisfiable Boolean formula} \}$$

Theorem

SAT *is* **NP***-complete.*

Proof of SAT $\in$ NP.

If $\pi \models \varphi$ we use $\langle \pi \rangle$ as certificate. □

Proof of **NP**-Hardness of SAT

Let $A \in \mathbf{NP}$. Let $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ be a deciding NTM with $L(M) = A$ and let p be a polynomial such that M takes at most $p(|w|)$ steps on any computation for any $w \in \Sigma^*$.

Construct a **P** reduction from A to SAT. On input w a Boolean formula φ_w that describes M's possible computations on w. M accepts w iff φ_w is satisfiable. The satisfying interpretation resolves the nondeterminism in the computation tree to arrive at an accepting branch of the computation tree.

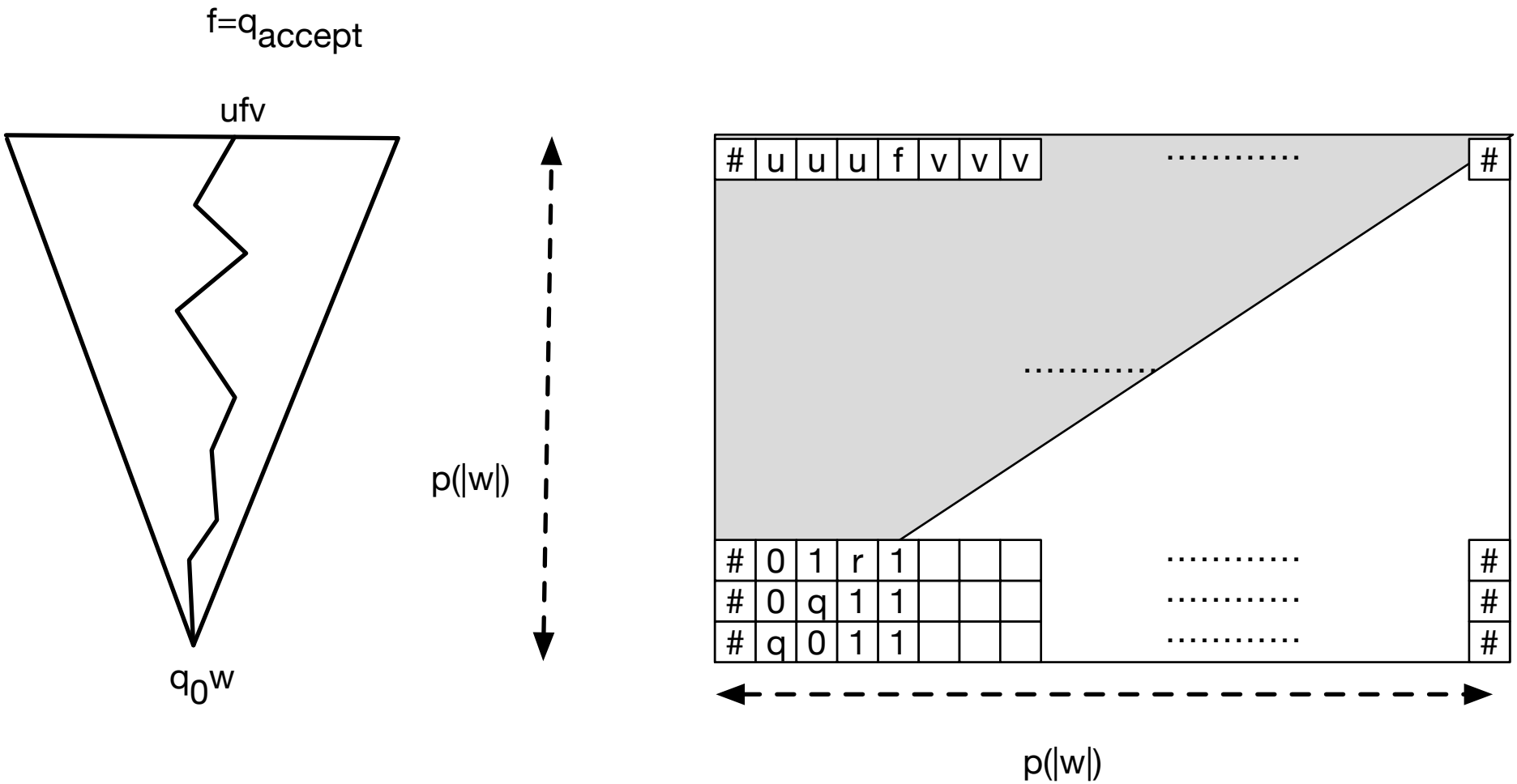
Remains to be done: define φ_w.

Proof of **NP**-Hardness of SAT cont.

Recall that M accepts w if an $n \leq p(|w|)$ and a sequence $(C_i)_{0<i\leq n}$ of configurations exist, where

1. $C_1 = q_0 w$,
2. each C_i can yield C_{i+1}, and
3. C_n is an accepting configuration.

Let $\Delta = Q \cup \Gamma \cup \{\#\}$. Each C_i can be represented as a $\#$-enclosed string over alphabet Δ no longer than $n + 3$.


f=q_accept
ufv
q_0w
p(|w|)
# u u u f v v v
#
# 0 1 r 1
# 0 q 1 1
# q 0 1 1
p(|w|)

φ_w

The Boolean formula φ_w shall represent *all* such sequences $(C_i)_{0<i\leq n}$ beginning with $q_0 w$.

$$\varphi_w = \varphi_{\text{cell}} \wedge \varphi_{\text{start}} \wedge \varphi_{\text{move}} \wedge \varphi_{\text{accept}}$$

. . . describes an n^2 grid using propositions

$$Prop = \{\ x_{i,k,s} \mid\ i,k \in \{1,\ldots,n\} \wedge s \in \Delta\ \}\ .$$

$$\varphi_{\text{cell}} = \bigwedge_{0<i,k\leq n} \left(\bigvee_{s\in\Delta} x_{i,k,s} \wedge \bigwedge_{s,t\in\Delta,s\neq t} \neg(x_{i,k,s} \wedge x_{i,k,t}) \right)$$

Row i in the grid corresponds to configuration C_i. Unused tape cells are blank.

Every grid cell contains exactly one symbol or a state.

φ_{start}

. . . specifies that the first row of the grid contains $q_0 w$ where $w = w_1 \dots w_{|w|}$:

$$\varphi_{\text{start}} = x_{1,1,\#} \wedge x_{1,2,q_0} \wedge \bigwedge_{2<i\leq|w|+2} x_{1,i,w_{i-2}} \wedge \bigwedge_{|w|+2<i\leq n-1} x_{1,i,\sqcup} \wedge x_{1,n,\#}$$

. . . ensures that C_i yields C_{i+1} by describing *legal* 2×3 windows of cells.

$$\varphi_{\text{move}} = \bigwedge_{0<i,k<n} \bigvee \begin{array}{|c|c|c|}\hline a_1 & a_2 & a_3 \\ \hline a_4 & a_5 & a_6 \\ \hline \end{array} \text{ is legal} \left(\begin{array}{lll} x_{i,k-1,a_1} & \wedge\, x_{i,k,a_2} & \wedge\, x_{i,k+1,a_3} \wedge \\ x_{i+1,k-1,a_4} & \wedge\, x_{i+1,k,a_5} & \wedge\, x_{i+1,k+1,a_6} \end{array} \right)$$

what is legal depends on the transition function δ.

Examples of legal 2x3 windows

a	b	c
a	b	c

$a,b,c \in \sum$

q	b	c
a	b	c

$a,b,c \in \sum$, **q** $\in Q$
(head moves into window from left)

c	**r**	b
q	a	b

$a,b,c \in \sum$, **q**,**r** $\in Q$
(**r** , c, R) $\in \delta$(**q**,a)
(head moves right)

φ_{accept}

. . . states that the accept state is reached:

$$\varphi_{\text{accept}} = \bigvee_{0<i,k\leq n} x_{i,k,q_{\text{accept}}}$$

Finally we check that the size of φ_w is polynomial in $|w|$ and that φ_w is constructable in polynomial time.

. . . states that the accept state is reached:

$$\varphi_{\text{accept}} = \bigvee_{0<i,k\leq n} x_{i,k,q_{\text{accept}}}$$

Finally we check that the size of φ_w is polynomial in $|w|$ and that φ_w is constructable in polynomial time.

— QED –

Cook's Theorem (SAT is **NP**-Complete)

Cook's theorem gives a "generic reduction" for every problem in **NP** to SAT. So SAT is as hard as any other problem in **NP**—it's **NP**-complete.

Cook's Theorem (SAT is **NP**-Complete)

Cook's theorem gives a "generic reduction" for every problem in **NP** to SAT. So SAT is as hard as any other problem in **NP**—it's **NP**-complete.

So, SAT is the granddaddy of all **NP**-complete problems.

Cook's Theorem (SAT is **NP**-Complete)

Cook's theorem gives a "generic reduction" for every problem in **NP** to SAT. So SAT is as hard as any other problem in **NP**—it's **NP**-complete.

So, SAT is the granddaddy of all **NP**-complete problems.

Many people have worked on the SAT problem, and there are now solvers (SAT solvers) for it that can solve problems up to thousands of variables in practice (though not polynomial time in theory).

People frequently translate **NP**-complete problems to propositional logic, and then attack them with these general solvers!

Outline

CSAT

CSAT is a special case of SAT.

CSAT

Input: A Boolean formula φ in cnf

Question: Is φ satisfiable?

where a Boolean formula is in *cnf* (for *conjunctive normal form*) if it is (also) generated by the grammar

$$\varphi \to (c) \mid (c) \wedge \varphi \qquad c \to \ell \mid \ell \vee c$$
$$\ell \to p \mid \neg p \qquad p \to x \mid y \mid \ldots$$

We call cs *clauses*, ℓs *literals*, and ps *propositions*.

Example

$(x \vee z) \wedge (y \vee z)$ is a cnf for the Boolean formula $(x \wedge y) \vee z$.

CSAT is **NP**-Complete

Clearly CSAT is in **NP** because we can use the same certificate for φ in cnf as we would for the same φ in SAT.

Giving a **P** reduction from SAT to CSAT is tricky.

A straight-forward translation of Boolean formulae into equivalent cnf may result in an exponential blow-up, meaning that this approach is useless.

CSAT is **NP**-Complete

Clearly CSAT is in **NP** because we can use the same certificate for φ in cnf as we would for the same φ in SAT.

Giving a **P** reduction from SAT to CSAT is tricky.

A straight-forward translation of Boolean formulae into equivalent cnf may result in an exponential blow-up, meaning that this approach is useless.

Instead, we recall a reduction f won't have to preserve *satisfaction*:

$$\forall \pi \, (\pi \models \varphi \quad \Leftrightarrow \quad \pi \models f(\varphi))$$

but merely *satisfiability*

$$\exists \pi \, (\pi \models \varphi) \quad \Leftrightarrow \quad \exists \pi \, (\pi \models f(\varphi))$$

meaning that we're free to choose different πs for the two sides.

3SAT

In fact, we can prove this for 3SAT, a special case of CSAT.

3SAT

Input: A formula φ in 3-CNF

Question: Is φ satisfiable?

where a Boolean formula is in *3-CNF* (for *3 literal conjunctive normal form*) if it is (also) generated by the grammar

$$\varphi \to (c) \mid (c) \wedge \varphi \qquad c \to \ell \mid \ell \vee \ell \mid \ell \vee \ell \vee \ell$$
$$\ell \to p \mid \neg p \qquad p \to x \mid y \mid \ldots$$

Example

$(x \vee y \vee z) \wedge (x \vee y \vee \neg z) \wedge (x \vee \neg y \vee z) \wedge (x \vee \neg y \vee \neg z)$ is a 3-CNF for the Boolean formula x.

$3\mathrm{SAT}$ is **NP**-Complete

Proof.

Clearly $3\mathrm{SAT}$ is in **NP** because we can use the same certificate for φ in 3-CNF as we would for the same φ in SAT (or CSAT).

Sipser prefers to adapt his **NP**-hardness proof for SAT to $3\mathrm{SAT}$ over giving a **P** reduction from SAT to $3\mathrm{SAT}$.

$3\mathrm{SAT}$ is **NP**-Complete

Proof.

Clearly $3\mathrm{SAT}$ is in **NP** because we can use the same certificate for φ in 3-CNF as we would for the same φ in SAT (or CSAT).

Sipser prefers to adapt his **NP**-hardness proof for SAT to $3\mathrm{SAT}$ over giving a **P** reduction from SAT to $3\mathrm{SAT}$.

We **P** reduce from SAT to $3\mathrm{SAT}$ instead, by translating arbitrary formulas into clauses with exactly three literals. □

Tseitin transformation

Theorem (Tseitin 1966)

For every propositional logic formula φ, we can construct in polynomial time a 3-CNF formula $f(\varphi)$ (with additional Boolean variables) such that $\varphi \in \mathrm{SAT}$ iff $f(\varphi) \in \mathrm{3SAT}$.

Proof.

Consider all the subformulas ψ of φ, and for each, let p_ψ be a new Boolean variable. Intuitively p_ψ represents the truth value of ψ.

$$f(\varphi) = p_\varphi \wedge \bigwedge_{\psi \text{ a subformula of } \varphi} \alpha(\psi)$$

where $\alpha(\psi)$ says that p_ψ is correctly related to the $p_{\psi'}$ for the *immediate* subformulas ψ' of ψ

□

ψ	$\alpha(\psi)$
p	$p_\psi \leftrightarrow p$
$\neg\psi_1$	$p_\psi \leftrightarrow \neg p_{\psi_1}$
$\psi_1 \wedge \psi_2$	$p_\psi \leftrightarrow (p_{\psi_1} \wedge p_{\psi_2})$
$\psi_1 \vee \psi_2$	$p_\psi \leftrightarrow (p_{\psi_1} \vee p_{\psi_2})$

Now notice that each $\alpha(\psi)$ involves at most three variables, and can be converted to a constant size equivalent 3-CNF by repeated application of boolean equivalences:

$$
\begin{aligned}
(\varphi \leftrightarrow \psi) &\equiv ((\varphi \rightarrow \psi) \wedge (\psi \rightarrow \varphi)) \\
(\varphi \rightarrow \psi) &\equiv ((\neg\varphi) \vee \psi) \\
\neg(\varphi \wedge \psi) &\equiv ((\neg\varphi) \vee (\neg\psi)) \\
\neg(\varphi \vee \psi) &\equiv ((\neg\varphi) \wedge (\neg\psi)) \\
\neg(\neg(\varphi)) &\equiv \varphi \\
\varphi \vee (\psi_1 \wedge \psi_2) &\equiv (\varphi \vee \psi_1) \wedge (\varphi \vee \psi_2)
\end{aligned}
$$

Example Tseitin Transformation

Example

$$f((x \wedge y) \vee \neg z) = p_{(x \wedge y) \vee \neg z} \quad \wedge \begin{cases} p_{(x \wedge y) \vee \neg z} \leftrightarrow (p_{(x \wedge y)} \vee p_{\neg z}) \quad \wedge \\ p_{(x \wedge y)} \leftrightarrow (p_x \wedge p_y) \quad \wedge \\ p_{\neg z} \leftrightarrow \neg p_z \quad \wedge \\ p_x \leftrightarrow x \quad \wedge \\ p_y \leftrightarrow y \quad \wedge \\ p_z \leftrightarrow z \end{cases}$$

Example Tseitin Transformation

Example

$$f((x \wedge y) \vee \neg z) = p_{(x \wedge y) \vee \neg z} \quad \wedge \begin{cases} p_{(x \wedge y) \vee \neg z} \leftrightarrow (p_{(x \wedge y)} \vee p_{\neg z}) \quad \wedge \\ p_{(x \wedge y)} \leftrightarrow (p_x \wedge p_y) \quad \wedge \\ p_{\neg z} \leftrightarrow \neg p_z \quad \wedge \\ p_x \leftrightarrow x \quad \wedge \\ p_y \leftrightarrow y \quad \wedge \\ p_z \leftrightarrow z \end{cases}$$

$$(p_{(x \wedge y)} \leftrightarrow p_x \wedge p_y) \equiv (p_{(x \wedge y)} \rightarrow (p_x \wedge p_y)) \wedge ((p_x \wedge p_y) \rightarrow p_{(x \wedge y)})$$

Example Tseitin Transformation

Example

$$f((x \wedge y) \vee \neg z) = p_{(x \wedge y) \vee \neg z} \quad \wedge \begin{cases} p_{(x \wedge y) \vee \neg z} \leftrightarrow (p_{(x \wedge y)} \vee p_{\neg z}) \ \wedge \\ p_{(x \wedge y)} \leftrightarrow (p_x \wedge p_y) \ \wedge \\ p_{\neg z} \leftrightarrow \neg p_z \ \wedge \\ p_x \leftrightarrow x \ \wedge \\ p_y \leftrightarrow y \ \wedge \\ p_z \leftrightarrow z \end{cases}$$

$$(p_{(x \wedge y)} \leftrightarrow p_x \wedge p_y) \equiv (p_{(x \wedge y)} \rightarrow (p_x \wedge p_y)) \wedge ((p_x \wedge p_y) \rightarrow p_{(x \wedge y)})$$

$$\begin{aligned} (p_{(x \wedge y)} \rightarrow (p_x \wedge p_y)) &\equiv (\neg p_{(x \wedge y)}) \vee (p_x \wedge p_y) \\ &\equiv (\neg p_{(x \wedge y)} \vee p_x) \wedge (\neg p_{(x \wedge y)} \vee p_y) \end{aligned}$$

Example Tseitin Transformation

Example

$$f((x \wedge y) \vee \neg z) = p_{(x \wedge y) \vee \neg z} \quad \wedge \begin{cases} p_{(x \wedge y) \vee \neg z} \leftrightarrow (p_{(x \wedge y)} \vee p_{\neg z}) \ \wedge \\ p_{(x \wedge y)} \leftrightarrow (p_x \wedge p_y) \ \wedge \\ p_{\neg z} \leftrightarrow \neg p_z \ \wedge \\ p_x \leftrightarrow x \ \wedge \\ p_y \leftrightarrow y \ \wedge \\ p_z \leftrightarrow z \end{cases}$$

$$(p_{(x \wedge y)} \leftrightarrow p_x \wedge p_y) \equiv (p_{(x \wedge y)} \rightarrow (p_x \wedge p_y)) \wedge ((p_x \wedge p_y) \rightarrow p_{(x \wedge y)})$$

$$\begin{aligned} (p_{(x \wedge y)} \rightarrow (p_x \wedge p_y)) &\equiv (\neg p_{(x \wedge y)}) \vee (p_x \wedge p_y) \\ &\equiv (\neg p_{(x \wedge y)} \vee p_x) \wedge (\neg p_{(x \wedge y)} \vee p_y) \end{aligned}$$

$$\begin{aligned} (p_x \wedge p_y) \rightarrow p_{(x \wedge y)} &\equiv \neg(p_x \wedge p_y) \vee p_{(x \wedge y)} \\ &\equiv \neg p_x \vee \neg p_y \vee p_{(x \wedge y)} \end{aligned}$$

VERTEX-COVER is NP-Complete

A *Vertex Cover* of an undirected graph G is a subset C of the nodes of G such that every edge contains node of C.

Vertex cover

Input: An undirected graph G and an integer k

Question: Does G have a k node vertex cover?

Vertex Cover is in **NP**. We show **NP**-completeness by proving $3\text{SAT} \leq_{\mathbf{P}}$ Vertex Cover on the whiteboard.

Clique is **NP**-Complete

A k-clique in an undirected graph is a set of k nodes such that there is an edge between each pair.

Clique

Input: An undirected graph G and an integer k

Question: Does G have a clique of size k?

We show **NP**-completeness in the tutorial.